INSTITUTO POLITÉCNICO DE SANTARÉM
Escola Superior de Gestão e Tecnologia

DOCUMENTAÇÃO DE TESTES

Sistema de Gestão Hoteleira (SGH)

Deliverable 5



Autores:
Guilherme Caçador - 240000675
Rodrigo Silva - 240000762

Engenharia de Software | Docente: Pedro Martins
2025/2026

1. Estratégia de Testes

A estratégia de garantia de qualidade do SGH baseia-se na Pirâmide de Testes, com foco massivo em testes unitários automatizados, complementados por testes de integração e testes de sistema. O objetivo é garantir cobertura mínima de 70% e detetar defeitos o mais cedo possível no ciclo de desenvolvimento.

1.1. Níveis de Teste

Testes Unitários:

Validam métodos e classes individuais de forma isolada, usando mocks para simular dependências externas. Foco na lógica de negócio dos serviços e entidades. Executados a cada commit.

Testes de Integração:

Validam a interação entre módulos, incluindo acesso à base de dados H2 em memória. Verificam que os repositórios JPA funcionam corretamente com as entidades.

Testes de Sistema:

Validam fluxos completos end-to-end, simulando cenários reais de utilização como criar reserva → check-in → check-out.

Testes de Aceitação:

Validam os critérios de aceitação das user stories, garantindo que o sistema satisfaz os requisitos do cliente.

1.2. Ferramentas de Teste

- JUnit 5: Framework de testes unitários padrão da indústria Java
- Mockito: Framework de mocking para isolar dependências em testes unitários
- JaCoCo: Análise de cobertura de código com relatórios HTML
- H2 Database: Base de dados em memória para testes de integração

1.3. Metodologias de Teste Aplicadas

- Particionamento por Equivalência: Agrupar inputs em classes equivalentes
- Análise de Valores Limite: Testar valores nos limites das classes
- Tabelas de Decisão: Testar combinações de condições de negócio
- Testes de Transição de Estado: Validar máquinas de estado (Quarto, Reserva)

1.4. Objetivos de Cobertura

Meta: Mínimo 70% de cobertura de código nos módulos implementados

Prioridade: Serviços de negócio (ServicoReserva, ServicoPrecos, ServicoCheckInOut) com cobertura >80%

Exclusões: Classes de configuração Spring, DTOs sem lógica, classes geradas

2. Casos de Teste

Esta secção documenta os 36 casos de teste desenvolvidos, organizados por módulo funcional. Cada caso inclui identificador único, descrição, pré-condições, passos de execução, resultado esperado e ligação ao requisito.

2.1. Módulo de Reservas (TC-001 a TC-011)

ID	Cenário	Pré-condições	Resultado Esperado	Req. Link	Estado
TC-001	Criar reserva válida	Quarto disponível, datas futuras, dados hóspede válidos	Reserva criada com ID único, estado CONFIRMADA	FR-002	Pass
TC-002	Reserva quarto ocupado	Quarto já reservado para as datas	QuartoIndisponivelException lançada	FR-002	Pass
TC-003	Reserva datas inválidas	Data saída anterior à entrada	DataInvalidaException lançada	FR-002	Pass
TC-004	Reserva data passada	Data entrada no passado	DataInvalidaException lançada	FR-002	Pass
TC-005	Consultar disponibilidade	3 quartos livres, 2 ocupados	Lista retorna apenas 3 quartos	FR-001	Pass
TC-006	Cancelar reserva válida	Reserva estado CONFIRMADA	Estado muda para CANCELADA	FR-003	Pass
TC-007	Cancelar já cancelada	Reserva estado CANCELADA	EstadoInvalidoException lançada	FR-003	Pass
TC-008	Cancelamento liberta quarto	Reserva cancelada	Quarto aparece em disponibilidade	FR-003	Pass
TC-009	Cancelamento <48h	Cancelamento 24h antes check-in	Alerta de taxa gerado	FR-003	Pass
TC-010	Notificação Observer	Observer registado no serviço	Observer.aoReservaCriada() invocado	FR-018	Pass
TC-011	Modificar reserva	Reserva existente, novas datas livres	Datas atualizadas, preço recalculado	FR-004	Pass

2.2. Módulo de Check-in/Check-out (TC-012 a TC-019)

ID	Cenário	Pré-condições	Resultado Esperado	Req. Link	Estado
TC-012	Check-in quarto pronto	Quarto estado PRONTO, reserva válida	Quarto estado OCUPADO, timestamp registado	FR-006	Pass
TC-013	Check-in quarto sujo	Quarto estado SUJO	QuartoNaoProntoException lançada	FR-006	Pass
TC-014	Check-in quarto manutenção	Quarto estado MANUTENCAO	QuartoNaoProntoException lançada	FR-006	Pass
TC-015	Check-in sem reserva	Nenhuma reserva para o hóspede	ReservaNaoEncontradaException	FR-006	Pass
TC-016	Check-out válido	Reserva estado CHECK_IN_FEITO	Fatura gerada, quarto estado SUJO	FR-007	Pass
TC-017	Check-out sem check-in	Reserva estado CONFIRMADA apenas	EstadoInvalidoException lançada	FR-007	Pass
TC-018	Fatura com extras	Consumos extra registados	Fatura inclui diárias + extras	FR-007, FR-008	Pass
TC-019	Cálculo de noites	Estadia de 3 noites a 100€	Fatura total = 300€	FR-007	Pass

2.3. Módulo de Preços - Strategy Pattern (TC-020 a TC-025)

ID	Cenário	Pré-condições	Resultado Esperado	Req. Link	Estado
TC-020	Preço época baixa	Quarto 100€, data Janeiro	Preço = 100€ (×1.0)	FR-014	Pass
TC-021	Preço época alta	Quarto 100€, data Agosto	Preço = 150€ (×1.5)	FR-014	Pass
TC-022	Desconto empresarial	Cliente tipo EMPRESARIAL	Preço = 85€ (×0.85)	FR-014	Pass
TC-023	Suite época alta	Suite 200€, data Agosto	Preço = 300€	FR-014	Pass
TC-024	Estratégia substituível	Trocar estratégia em runtime	Cálculo usa nova estratégia (LSP)	FR-014	Pass
TC-025	Estratégias combinadas	Época alta + desconto empresa	(100×1.5)×0.85 = 127.50€	FR-014	Pass

2.4. Módulo de Limpeza e Manutenção (TC-026 a TC-030)

ID	Cenário	Pré-condições	Resultado Esperado	Req. Link	Estado
TC-026	Marcar quarto limpo	Quarto estado SUJO	Estado muda para PRONTO	FR-009	Pass
TC-027	Notificação limpeza	ObservadorQuarto registado	Observer notificado da mudança	FR-009	Pass
TC-028	Lista quartos sujos	3 quartos SUJO na BD	Lista retorna 3 elementos	FR-010	Pass
TC-029	Reportar avaria	Quarto disponível	Estado MANUTENCAO, ticket criado	FR-011	Pass
TC-030	Encerrar manutenção	Ticket aberto para quarto	Quarto estado SUJO, ticket fechado	FR-012	Pass

2.5. Módulo de Autenticação (TC-031 a TC-036)

ID	Cenário	Pré-condições	Resultado Esperado	Req. Link	Estado
TC-031	Login válido	Utilizador existe, password correta	Sessão criada, perfil retornado	FR-017	Pass
TC-032	Login password errada	Utilizador existe, password incorreta	AutenticacaoException lançada	FR-017	Pass
TC-033	Login user inexistente	Username não existe na BD	AutenticacaoException lançada	FR-017	Pass
TC-034	Acesso não autorizado	User LIMPEZA acede relatórios	AcessoNegadoException lançada	FR-017	Pass
TC-035	Password BCrypt	Nova password a guardar	Hash BCrypt gerado, não texto claro	NFR-004	Pass
TC-036	Bloqueio força bruta	5 tentativas falhadas seguidas	Conta bloqueada temporariamente	NFR-006	Pass

3. Testes White-Box

Os testes white-box focam-se na estrutura interna do código, garantindo cobertura de statements, branches e paths. São essenciais para validar a lógica de decisão e detetar código morto.

3.1. Análise de Valores Limite - Quantidade de Reserva

Cenário: O sistema aceita reservas de 1 a 30 noites. Testamos os valores limite.

```
@ParameterizedTest
@ValueSource(ints = {0, 1, 15, 30, 31})
void testeValidarDuracaoReserva(int noites) {
    if (noites == 0 || noites == 31) {
        assertThrows(DuracaoInvalidaException.class,
            () -> servicoReserva.validarDuracao(noites));
    } else {
        assertDoesNotThrow(() -> servicoReserva.validarDuracao(noites));
    }
}
```

3.2. Cobertura de Branches - Cálculo de Preço Sazonal

Cenário: Testar todos os branches do método getMultiplicadorAtual().

```
@Test
void testeMultiplicadorEpocaAlta() {
    // Branch: mes >= 6 && mes <= 9
    LocalDate agosto = LocalDate.of(2025, 8, 15);
    assertEquals(1.5, estrategia.getMultiplicador(agosto));
}

@Test
void testeMultiplicadorEpocaMedia() {
    // Branch: mes == 12 || mes <= 2
    LocalDate dezembro = LocalDate.of(2025, 12, 20);
    assertEquals(1.2, estrategia.getMultiplicador(dezembro));
}

@Test
void testeMultiplicadorEpocaBaixa() {
    // Branch: else (default)
    LocalDate marco = LocalDate.of(2025, 3, 10);
    assertEquals(1.0, estrategia.getMultiplicador(marco));
}
```

3.3. Cobertura de Paths - Transição de Estado do Quarto

Cenário: Testar todos os paths possíveis na máquina de estados do Quarto.

```
@Test
void testePath_Pronto_CheckIn_Ocupado() {
    quarto.setEstado(EstadoQuarto.PRONTO);
    servicoCheckIn.realizarCheckIn(quarto, reserva);
    assertEquals(EstadoQuarto.OCUPADO, quarto.getEstado());
}

@Test
void testePath_Ocupado_CheckOut_Sujo() {
    quarto.setEstado(EstadoQuarto.OCUPADO);
    servicoCheckOut.realizarCheckOut(quarto, reserva);
    assertEquals(EstadoQuarto.SUJO, quarto.getEstado());
}

@Test
void testePath_Sujo_Limpeza_Pronto() {
    quarto.setEstado(EstadoQuarto.SUJO);
    servicoLimpeza.marcarLimpo(quarto);
    assertEquals(EstadoQuarto.PRONTO, quarto.getEstado());
}
```

3.4. Verificação de Interações com Mockito

Cenário: Verificar que o Observer é notificado exatamente uma vez ao criar reserva.

```
@Test
void testeObserverNotificado() {
    // Arrange
    ObservadorReserva mockObserver = mock(ObservadorReserva.class);
    servicoReserva.adicionarObservador(mockObserver);

    // Act
    Reserva reserva = servicoReserva.criar(dadosValidos);

    // Assert - verificar interação
    verify(mockObserver, times(1)).aoReservaCriada(reserva);
    verifyNoMoreInteractions(mockObserver);
}
```

3.5. Teste de Exceções Esperadas

Cenário: Garantir que exceções apropriadas são lançadas em condições de erro.

```
@Test
void testeExcecaoQuartoIndisponivel() {
    // Arrange - quarto já reservado
    when(repositorio.verificarDisponibilidade(any(), any(), any()))
        .thenReturn(false);

    // Act & Assert
    QuartoIndisponivelException ex = assertThrows(
        QuartoIndisponivelException.class,
        () -> servicoReserva.criar(dadosReserva)
    );

    assertTrue(ex.getMessage().contains("não disponível"));
}
```

4. Relatório de Cobertura

A cobertura de código é medida através do JaCoCo, integrado no build Maven. O objetivo é manter cobertura mínima de 70% nos módulos de negócio.

Módulo	Linhas	Branches	Estado
com.sgh.servico	93%	83%	✓ Acima do objetivo
com.sgh.modelo	96%	92%	✓ Acima do objetivo
com.sgh.fabrica	89%	100%	✓ Acima do objetivo
com.sgh.estrategia	97%	95%	✓ Acima do objetivo
com.sgh.repositorio	99%	85%	✓ Acima do objetivo
com.sgh.observador	100%	100%	✓ Acima do objetivo
com.sgh.modelo.enums	100%	n/a	✓ Acima do objetivo
Com.sgh.excecao	100%	n/a	✓ Acima do objetivo
TOTAL PROJETO	**96%**	**88%**	**✓ Objetivo cumprido (>70%)**

Tabela: Cobertura de Código por Módulo